BEAN BEACON

Final Project Documentation

Version 3.0 — Official Consolidated Reference

A full-stack café discovery platform combining interactive mapping, community reviews, intelligent recommendations, and production-grade observability.

Document Type	Technical Reference
Architecture	3-Tier (React · Express · MongoDB)
Status	Production-Ready
Classification	Official Consolidated Reference

1. Table of Contents

1. Executive Summary & Project Overview
2. Key Features
3. Technology Stack
4. System Architecture
5. DevOps & CI/CD Pipeline
6. Observability & Monitoring
7. Deployment Guide
8. API Reference Overview
9. Final Deliverables

1. Executive Summary & Project Overview

What is Bean Beacon?

Bean Beacon is a modern, full-stack web application designed as an intelligent café discovery platform. It empowers users to explore coffee shops, specialty cafés, and related venues through an interactive map experience, community-driven ratings and reviews, personalized recommendations, and curated management tools. The platform is optimized for location-aware discovery (with strong support for Tunisia and surrounding regions) while remaining flexible for global expansion.

Core Value Proposition

- **Café Discovery** — Real-time, location-based search powered by OpenStreetMap data, enriched with user-generated content and intelligent filtering.
- **Community Engagement** — Ratings, reviews, and favorites that create a living knowledge base of café experiences.
- **Personalized Recommendations** — Preference-driven ranking engine that matches user tastes in drinks, atmosphere, and location.
- **Operational Excellence** — Secure authentication, role-based administration, photo management, and a production-ready observability stack.

High-Level Architecture Summary

Bean Beacon follows a clean 3-tier architecture:

- **Presentation Layer**: React + TypeScript single-page application with Leaflet maps, Tailwind CSS, and shadcn/ui components.
- **Application Layer**: Node.js / Express REST API with TypeScript, JWT authentication, and business logic services.
- **Data Layer**: MongoDB (via Mongoose) for persistent storage of users, cafés, favorites, and ratings.

External intelligence is provided by Nominatim (geocoding) and Overpass API (OpenStreetMap café data), while the entire stack is containerized with Docker Compose and continuously validated through a GitLab CI pipeline.

2. Key Features

• **Interactive Map Experience** — Leaflet-powered map with custom coffee-cup markers, user location awareness, and smooth navigation focused on the Tunisia region and beyond.

• **Location-Based Café Discovery** — Real-time queries against OpenStreetMap via the Overpass API, with radius-based search, caching, and enrichment.

• **Smart Recommendations Engine** — Preference matching across drink type, atmosphere, and location with transparent ranking explanations.

• **Community Ratings & Reviews** — Full CRUD for café ratings (1–5 stars + text reviews) with ownership controls.

• **Favorites System** — Authenticated users can save and manage personal café collections.

• **Secure JWT Authentication** — Registration, login, and session management with role support (user / admin).

• **Admin Dashboard** — Comprehensive management of users, cafés, and reviews with elevated privileges.

• **Café Management** — Create, update, soft-status control, and photo upload capabilities for café owners and administrators.

• **Attribute Filtering** — Multi-dimensional filters for coffee types, atmosphere, and amenities.

• **Search & Category Navigation** — Fast text search and category-based browsing.

• **Photo Upload Support** — Secure image handling for café profiles with static serving.

• **Health & Metrics Endpoints** — Built-in health checks and Prometheus metrics for operational visibility.

• **Production-Ready Observability** — Prometheus, Grafana, cAdvisor, Node Exporter, and Uptime Kuma integration.

• **Fully Containerized Environments** — Dedicated Docker Compose configurations for development and production.

3. Technology Stack

Category	Technologies
Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui, Radix UI, React Router, TanStack Query, Leaflet / React-Leaflet, Lucide icons
Backend	Node.js, Express, TypeScript, Mongoose, JWT (jsonwebtoken), bcryptjs, Multer, prom-client, Axios, node-cache, express-validator
Database	MongoDB 7
External APIs	Overpass API (OpenStreetMap), Nominatim (OpenStreetMap geocoding)
DevOps	Docker, Docker Compose, GitLab CI/CD, Nginx (production frontend)
Observability	Prometheus, Grafana, cAdvisor, Node Exporter, Uptime Kuma, prom-client
Testing	Vitest (backend)
Tooling	ESLint, TypeScript compiler, tsx

4. System Architecture

3-Tier Architecture

Bean Beacon implements a classic, maintainable 3-tier design:

1. **Frontend (React SPA)** — Runs on Vite in development and is served by Nginx in production. Communicates exclusively with the backend via a clean REST API (VITE_API_URL).
2. **Backend (Express API)** — Handles authentication, business logic, data enrichment from external services, file uploads, and metrics exposure.
3. **Database (MongoDB)** — Stores users, café records, favorites, and ratings with Mongoose schemas and proper indexing support.

DevOps & System Architecture Diagram

The diagram below illustrates the complete Bean Beacon DevOps architecture — from developer workflow through the GitLab CI/CD pipeline to the production environment and full observability stack.



Figure 1 — Bean Beacon DevOps Architecture: From code to production — Automated, secure, observable.

External API Roles

• **Overpass API** — Primary source of café/amenity data from OpenStreetMap. Queries are cached (1-hour TTL) and use multiple resilient endpoints.

• **Nominatim** — Converts human-readable addresses into geographic coordinates for café creation and enrichment. Respects User-Agent requirements and caching.

• **Leaflet + OpenStreetMap Tiles** — Client-side map rendering and interaction (no Google Maps dependency required for core functionality).

5. DevOps & CI/CD Pipeline

GitLab CI Pipeline Stages

The pipeline (.gitlab-ci.yml) is organized into four progressive stages that enforce quality gates:

Stage	Jobs	Purpose
Validate	backend:typecheck, frontend:lint, frontend:typecheck, frontend:build	Type safety, linting, and successful production builds
Test	backend:test	Vitest unit tests (non-watch mode)
Security	backend:audit, frontend:audit, gitleaks	Dependency vulnerability scanning and secret detection
Build	backend:docker-build, frontend:docker-build	Production Docker image construction tagged with commit SHA

All jobs target a self-hosted runner tagged bean-beacon. Later stages execute only after earlier stages succeed, ensuring that only validated, tested, and secure code proceeds to image builds.

Docker Compose Environments

- **Development** (docker-compose.yml) — Hot-reload volumes, exposed ports for backend (3000), frontend (8081), Prometheus (9090), Grafana (3001), Uptime Kuma (3002), cAdvisor (8082), and Node Exporter (9100). Includes health checks and service dependency ordering.
- **Production** (docker-compose.prod.yml) — Optimized multi-stage builds, internal networking, persistent volumes for MongoDB and uploads, and Nginx-served frontend on port 80. Requires explicit JWT_SECRET injection.

6. Observability & Monitoring

Bean Beacon ships with a complete observability stack:

- **Prometheus** — Scrapes the backend /metrics endpoint (custom + default Node.js metrics via prom-client), cAdvisor, and Node Exporter on a 15-second interval.
- **Custom Backend Metrics** — Histogram of HTTP request duration and counter of total requests, labeled by method, route, and status code. Prefix: bean_beacon_.
- **Grafana** — Pre-provisioned with Prometheus as a data source for dashboard creation.
- **Uptime Kuma** — Independent uptime and status monitoring service.
- **cAdvisor & Node Exporter** — Container and host-level resource metrics.

The /metrics endpoint is intentionally placed outside the /api prefix so that production Nginx proxy rules remain clean.

7. Deployment Guide

Development Environment

```
# From the repository root
docker compose up --build

# Services available at:
# Frontend          → http://localhost:8081
# Backend API       → http://localhost:3000
# Prometheus        → http://localhost:9090
# Grafana           → http://localhost:3001
# Uptime Kuma       → http://localhost:3002
```

Production Environment

```
# Ensure JWT_SECRET is set in the environment or a .env file
$env:JWT_SECRET = "your-strong-production-secret"
docker compose -f docker-compose.prod.yml up --build -d

# Application → http://localhost (port 80)
# Backend remains internal; Nginx proxies /api requests.
```

Additional Automation

The repository is structured for infrastructure automation. Deployment can be further streamlined with infrastructure-as-code tooling (pyinfra or equivalent) that applies the same Docker Compose definitions across target hosts, manages secrets, and performs rolling updates.

8. API Reference Overview

All authenticated endpoints expect an Authorization: Bearer <token> header. Responses follow a consistent { success, data | error } envelope.

Resource	Method & Path	Description	Auth
Auth	POST /api/auth/register	Create new user account	Public
	POST /api/auth/login	Authenticate and receive JWT	Public
	GET /api/auth/me	Retrieve current user profile	Bearer JWT
Cafés	GET /api/cafes	List nearby cafés (location + filters)	Optional
	GET /api/cafes/search	Text search	Optional
	GET /api/cafes/all	Flat list of all cafés	Optional
	GET /api/cafes/:id	Café detail	Optional
	POST /api/cafes	Create café	Authenticated
	PATCH /api/cafes/:id	Update café details	Authenticated
	PATCH /api/cafes/:id/status	Update café status	Admin
	POST /api/cafes/:id/photo	Upload café photo	Authenticated
	DELETE /api/cafes/:id	Delete café	Authenticated
	POST /api/cafes/recommend	Preference-based recommendations	Public
Favorites	GET /api/favorites	List user favorites	Authenticated
	POST /api/favorites/:cafeId	Add favorite	Authenticated
	DELETE /api/favorites/:cafeId	Remove favorite	Authenticated
	GET /api/favorites/:cafeId/check	Check favorite status	Optional
Ratings	GET /api/cafes/:cafeId/ratings	List ratings for a café	Optional
	POST /api/cafes/:cafeId/ratings	Add rating/review	Authenticated
	PUT /api/cafes/:cafeId/ratings/:ratingId	Update own rating	Authenticated
	DELETE /api/cafes/:cafeId/ratings/:ratingId	Delete own rating	Authenticated
Admin	GET /api/admin/reviews	All reviews (moderation)	Admin
	GET /api/admin/users	List users	Admin
	POST /api/admin/users	Create user (with role)	Admin
	PATCH /api/admin/users/:id	Update user	Admin
	PATCH /api/admin/cafes/:id	Admin café update	Admin
System	GET /api/health	Health check	Public
	GET /metrics	Prometheus metrics	Public

9. Final Deliverables

Deliverable	Location / Path	Description
Backend Source	bean-beacon-backend/src/	Express + TypeScript API, models, services, controllers
Frontend Source	bean-beacon-frontend/src/	React + TypeScript SPA with map, recommender, admin UI
Development Compose	docker-compose.yml	Full local stack with hot reload and monitoring
Production Compose	docker-compose.prod.yml	Optimized production deployment
CI/CD Pipeline	.gitlab-ci.yml	Validate → Test → Security → Build stages
Prometheus Config	monitoring/prometheus.yml	Scrape targets for backend, cAdvisor, Node Exporter
Grafana Provisioning	monitoring/grafana/provisioning/	Datasource auto-configuration
Backend Dockerfile (prod)	bean-beacon-backend/Dockerfile	Multi-stage production image
Backend Dockerfile (dev)	bean-beacon-backend/Dockerfile.dev	Development image with volume support
Frontend Dockerfile (prod)	bean-beacon-frontend/Dockerfile	Nginx-served static build
Frontend Dockerfile (dev)	bean-beacon-frontend/Dockerfile.dev	Vite development server
Environment Template	bean-beacon-backend/.env.example	Required configuration variables
Unit Tests	bean-beacon-backend/tests/	Vitest suites (including recommendation engine)
Nginx Configuration	bean-beacon-frontend/nginx/default.conf	Production reverse-proxy and SPA routing

Bean Beacon v3.0 — Production-ready, observable, and designed for continuous enhancement.